AI Usage Log

Task SCRUM-24: Create the lint configuration

Purpose of AI Use: To help setup the lint configuration file (eslint.config.js)

Prompt: eslint.config.js setup for node.js and express.js environment

Response:

Here is the standard **eslint.config.js** file tailored for a modern **Node.js and Express.js** backend using the current **ESLint Flat Config** format (v9+).

1. Install Dependencies
Make sure you have installed the core flat-config packages and the Node environment globals via your terminal:

```
npm install --save-dev eslint @eslint/js globals
```

*If your Express project uses **TypeScript**, also run:* npm install --save-dev typescript-eslint

2. The eslint.config.js Setup
Option A: For Node.js + Express (JavaScript)
This configuration applies base JavaScript recommended rules and explicitly enables Node.js-specific global variables (like process, __dirname, and require).

```
import js from "@eslint/js";
import globals from "globals";

/** @type {import('eslint').Linter.Config[]} */
export default [
  // 1. Global ignores (replaces .eslintignore)
  {
    ignores: ["node_modules/", "dist/", "build/", "coverage/"]
  },

  // 2. Base JS configuration
  js.configs.recommended,

  // 3. Node/Express Environment Specifics
  {
    files: ["**/*.{js,cjs,mjs}"],
    languageOptions: {
      ecmaVersion: "latest",
      sourceType: "module", // Use "commonjs" if your project uses require() instead of
import
```

```
    globals: {
      ...globals.node,  // Enables node globals like process, require, module
      ...globals.es2021
    },
   },
   rules: {
    // Backend specific adjustments
    "no-console": "off",          // Off because console.log/error is standard for server
logging
    "no-unused-vars": ["warn", {
     "argsIgnorePattern": "^_",    // Allows unused Express args like next or _req
     "varsIgnorePattern": "^_"
    }],
    "prefer-const": "error",
    "no-process-exit": "off"       // Off to allow process.exit() if you need to shut down the
server safely
   },
 },
];
```

Option B: For Node.js + Express + TypeScript
If you are building your Express API with TypeScript, use this type-aware approach:

```
import js from "@eslint/js";
import globals from "globals";
import tseslint from "typescript-eslint";

export default tseslint.config(
 {
  ignores: ["node_modules/", "dist/", "build/", "coverage/"]
 },
 js.configs.recommended,
 ...tseslint.configs.recommended,
 {
  files: ["**/*.ts"],
  languageOptions: {
   globals: {
    ...globals.node,
   },
  },
  rules: {
   "no-console": "off",
   "@typescript-eslint/no-unused-vars": ["warn", {
    "argsIgnorePattern": "^_", // Allows unused Express 'next' or '_req' variables
```

```
      "varsIgnorePattern": "^_"
    }],
  },
 }
);
```

AI-Suggested Content: Generated the lint configuration file that uses the same technologies as the project

Validation: Tested manually the script

Decision: Accepted as the generated pipeline configuration file worked properly

Reflection: AI helped understand how the pipeline configuration file works.

Responsible Person: Johnny Quach

Task SCRUM-24: Change the project structure

Purpose of AI Use: To help change the project structure

Prompt: what is the project structure of the repo consists of the client folder and server folder

Response:

When your repository is split into a **client** folder (frontend, like React) and a **server** folder (backend, like Node.js), it is called a monorepo.
In this setup, **each folder acts like its own mini-project** with its own package.json and linting rules.
Here is exactly what your project structure should look like to make everything work seamlessly:

📁 **The Complete Project Structure**

```
your-repo-name/
 ├── .github/
 │   └── workflows/
 │       └── ci.yml          <-- Your pipeline configuration
 │
 ├── client/                 <-- FRONTEND (React, Vue, Next.js, etc.)
 │   ├── src/
 │   ├── eslint.config.js    <-- Frontend styling rules (e.g., React rules)
 │   └── package.json        <-- Frontend scripts ("lint": "eslint src/")
 │
 └── server/                 <-- BACKEND (Node.js, Express, etc.)
     ├── src/
     ├── eslint.config.js    <-- Backend styling rules (e.g., Node.js rules)
     └── package.json        <-- Backend scripts ("lint": "eslint src/")
```

⚠️ **The Pipeline Problem (And How to Fix It)**

Because you now have *two separate* package.json files hidden inside subfolders, running a simple npm ci or npm run lint at the very root of your project will **fail**—Node won't find the files.
You have two easy ways to handle this inside your GitHub pipeline YAML file:

Option A: Tell npm which folder to target (Easiest)

You can use the --prefix flag to tell npm exactly which folder to enter to run the command:

```
- name: Install and Lint Client
  run: |
    npm ci --prefix client
    npm run lint --prefix client
```

```
- name: Install and Lint Server
  run: |
    npm ci --prefix server
    npm run lint --prefix server
```

Option B: Change the "working directory"
You can tell GitHub to step inside the folder first before running any commands:

```
- name: Run Client Lint
  working-directory: ./client
  run: |
    npm ci
    npm run lint
```

AI-Suggested Content: Proposed a project structure to follow by suggesting splitting the codebase into 2 directories: client and server. Proposed a solution for the pipeline configuration.

Validation: The pipeline configuration was manually tested and the project structure was reviewed

Decision: Accepted

Reflection: AI helped better understand how to organize the code and how the pipeline configuration works.

Responsible Person: Johnny Quach

Task SCRUM-24: Properly configure the src folder in the server directory

Purpose of AI Use: To properly configure the src folder after changing the project structure

Prompt: what should the src folder contain?

Response:

The **src** (source) folder should contain all of your original, raw application code that you write by hand.
Because you have a separate client and server, **each folder will have its own independent src directory**, and what goes inside them is completely different.
Here is a breakdown of what typically belongs in each src folder:

🎨 **Inside client/src/ (Frontend)**
This folder holds everything responsible for what the user sees and interacts with on their screen (e.g., React, Vue, or Angular).

- **components/**: Small, reusable UI pieces like buttons, navigation bars, cards, and input forms.
- **pages/** or **views/**: The main page layouts of your app (e.g., Home.jsx, Dashboard.jsx, Login.jsx).
- **assets/**: Images, logos, icons, and global CSS styling files.
- **main.jsx** (or index.js): The main entry point that boots up the frontend application and injects it into the browser.

⚙️ **Inside server/src/ (Backend)**
This folder holds the logic that runs behind the scenes on your server (e.g., Node.js with Express). It handles data, security, and talking to your database.

- **routes/**: Files that define your URL endpoints (e.g., authRoutes.js handles login URLs, productRoutes.js handles data URLs).
- **controllers/**: The actual logic and functions that run when someone hits a route (e.g., fetching data from a database or verifying a password).
- **models/**: The blueprints or schemas for your database data (e.g., defining exactly what a "User" or an "Order" profile looks like).
- **middleware/**: Security and helper checks that run *before* hitting your data (e.g., checking if a user is logged in).
- **index.js** (or app.js, server.js): The main entry point that starts up your backend server and listens for requests.

AI-Suggested Content: Described what the src folder in both directories should contain.

Validation: The proposed solution was reviewed

Decision: Accepted

Reflection: AI helped better understand how to organize the code within both directories.

Responsible Person: Johnny Quach

Task SCRUM-24: Properly configure the src folder in the server directory

Purpose of AI Use: To put the config folder and the queries folder in the correct folder

Prompt: what if you have a config folder and a queries folder in the server directory ?

Response:

If you have a **config** folder and a **queries** folder inside your server directory, your backend architecture is getting more organized and production-ready!
These folders split out two critical backend tasks: **handling environment settings** and **talking to your database**.
Here is exactly what belongs inside those folders and where they fit into your server layout:

📁 **Updated Server Project Structure**

You can place these folders either inside your server/src/ directory (highly recommended to keep code bundled together) or directly at the root of the server/ directory.

```
server/
├── src/
│   ├── config/     <-- Database connections & environment variables
│   │   └── db.js
│   ├── queries/    <-- SQL or database query definitions
│   │   └── userQueries.js
│   ├── routes/
│   ├── controllers/
│   └── index.js
├── eslint.config.js
└── package.json
```

⚙️ **1. What belongs in the config folder?**

The config folder should hold files that **initialize third-party tools, manage secret credentials, or set up connections**.

- **db.js (or database.js):** The file that initializes your database client (like MongoDB/Mongoose, PostgreSQL/pg-pool, or MySQL) and connects your server to your database.
- **passport.js:** Configurations for user authentication or login strategies.
- **env.js:** A central file that safely exports your system's environment variables (like secret keys, ports, and API credentials) using tools like dotenv.

🔍 **2. What belongs in the queries folder?**

The queries folder is used to separate your raw database commands away from your main business logic. This keeps your files clean and highly reusable.

- **If using SQL (PostgreSQL/MySQL):** This folder will hold files containing raw SQL query strings (e.g., SELECT * FROM users WHERE id = $1;).

- **If using a database helper (like Knex or an ORM):** It holds functions dedicated strictly to fetching data.
- **Why have it?** Instead of writing a massive 20-line database query inside your controller, you import a tidy, one-line function like getUserById(id) straight from your queries/userQueries.js file.

🚨 **How this impacts your Pipeline and Linter**

Adding these folders means your **Linter (npm run lint) has more files to check**.
As long as these folders live inside your server/src/ directory (or wherever your package.json script points, like eslint .), ESLint will automatically scan your database settings and your SQL files for spelling bugs, dead code, or unhandled errors.

AI-Suggested Content: Described where the two folders should be place and informed on how the pipeline may have to be changed.

Validation: The proposed solution was reviewed

Decision: Accepted

Reflection: AI helped better understand how to organize the code within the src folder.

Responsible Person: Johnny Quach

Task SCRUM-24: Set up the configuration for unit testing

Purpose of AI Use: Create the configuration file for unit testing

Prompt: github repo with client and server structure. already have the pipeline yaml file. setup the unit test files for the client directory in react framework and the server directory in node.js and express.js environment

Response:

Here is the structured setup for adding unit tests to your **React client** and **Node.js/Express server**. This setup uses industry-standard testing frameworks (**Jest** and **React Testing Library** for the client, and **Jest** with **Supertest** for the server) that integrate seamlessly into existing CI/CD pipelines.

📁 Recommended Directory Structure

Add the new test files (marked with 🆕) into your existing repository layout:

```
my-repo/
├── .github/
│   └── workflows/
│       └── pipeline.yaml
├── client/                 # React Application
│   ├── src/
│   │   ├── components/
│   │   │   ├── Button.js
│   │   │   └── 🆕 Button.test.js
│   ├── 🆕 jest.config.js
│   └── package.json
└── server/                 # Node.js / Express Application
    ├── src/
    │   ├── app.js
    │   └── routes/
    │       ├── auth.js
    │       └── 🆕 auth.test.js
    ├── 🆕 jest.config.js
    └── package.json
```

💻 1. Client Setup (React)

Step A: Install Dependencies

Navigate to your client/ directory and install the required testing libraries:

```
cd client
npm install --save-dev jest babel-jest @testing-library/react @testing-library/jest-dom
@testing-library/user-event jest-environment-jsdom
```

Step B: Configuration (client/jest.config.js)

Create a Jest configuration file specifically for the browser environment:

```
module.exports = {
  testEnvironment: 'jsdom',
  setupFilesAfterEnv: ['@testing-library/jest-dom'],
  moduleNameMapper: {
    '\\.(css|less|scss|sass)$': 'identity-obj-proxy', // Mocks CSS imports
  },
};
```

Step C: Add Test Script (client/package.json)

Add the test command to your scripts block:

```
"scripts": {
  "test": "jest --passWithNoTests"
}
```

Step D: Example Unit Test (client/src/components/Button.test.js)

```
import React from 'react';
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import Button from './Button'; // Adjust based on your component path

describe('Button Component', () => {
  test('renders button with correct text', () => {
    render(<Button label="Click Me" />);
    const buttonElement = screen.getByRole('button', { name: /click me/i });
    expect(buttonElement).toBeInTheDocument();
  });

  test('calls onClick handler when clicked', async () => {
    const handleClick = jest.fn();
    render(<Button label="Click Me" onClick={handleClick} />);

    const buttonElement = screen.getByRole('button', { name: /click me/i });
    await userEvent.click(buttonElement);

    expect(handleClick).toHaveBeenCalledTimes(1);
  });
});
```

🌐 **2. Server Setup (Node.js & Express)**

Step A: Install Dependencies

Navigate to your server/ directory and install Jest along with Supertest (used for testing HTTP requests):

```
cd ../server
npm install --save-dev jest supertest
```

Step B: Configuration (server/jest.config.js)
Create a Jest config optimized for the Node environment:

```
module.exports = {
  testEnvironment: 'node',
  verbose: true,
  forceExit: true, // Useful if your app leaves open handles (like DB connections)
  clearMocks: true,
};
```

Step C: Add Test Script (server/package.json)
Add the test command to your server scripts block:

```
"scripts": {
  "test": "jest --runInBand --passWithNoTests"
}
```

(Note: --runInBand forces tests to run sequentially, which prevents database collisions if your server tests touch a real or mocked DB).

Step D: Example API Test (server/src/routes/auth.test.js)

```
const request = require('supertest');
const app = require('../app'); // Path to your Express app instance (before app.listen)

describe('Auth Endpoints', () => {
  describe('POST /api/login', () => {
    test('should return 200 and a token for valid credentials', async () => {
      const res = await request(app)
        .post('/api/login')
        .send({
          email: 'user@example.com',
          password: 'password123'
        });

      expect(res.statusCode).toEqual(200);
      expect(res.body).toHaveProperty('token');
    });

    test('should return 401 for invalid credentials', async () => {
      const res = await request(app)
```

```
      .post('/api/login')
      .send({
        email: 'wrong@example.com',
        password: 'wrongpassword'
      });

    expect(res.statusCode).toEqual(401);
  });
 });
});
```

(Tip: Make sure your app.js exports the app instance using module.exports = app; without calling app.listen() inside the same file, so Supertest can manage the server lifecycle during testing).

AI-Suggested Content: Generated configuration files and unit test files for both the client and server directory.

Validation: The configuration files were tested, and the unit test files were compared with the required functionalities.

Decision: Partially accepted

Reflection: AI helped better understand what configuration files are used for.

Responsible Person: Johnny Quach

Task SCRUM-24: Making lint and unit testing compatible

Purpose of AI Use: To make lint and unit testing compatible

Prompt: "npm run test:unit" and "npm run lint" are incompatible because of the "type":"commonjs" and "type":"module"

Response:

This conflict occurs because Node.js requires a project to declare a single default module system ("type": "commonjs" or "type": "module") inside **package.json, but traditional configurations for ESLint and Jest require opposite systems.**

For example, older versions of ESLint configurations (.eslintrc.js) use CommonJS (module.exports), while your test runner or project code might be written using modern ES Modules (import/export).

You can resolve this issue using two different methods:

Option 1: Keep "type": "module" (Recommended for Modern Projects)
If you want to keep using modern ES Modules ("type": "module") for your project and your unit tests, you just need to update your lint configuration file to use the explicit CommonJS extension.

1. Open your project root directory.
2. Rename **.eslintrc.js** to **.eslintrc.cjs**.
   (The .cjs extension forces Node.js to treat that specific file as CommonJS, ignoring the "type": "module" rule in your package.json.)

Option 2: Remove "type": "module" and use .mjs or a compiler
If your unit test runner (like Jest) is throwing syntax errors because it struggles to parse ES Modules natively, you can invert the strategy:

1. Remove "type": "module" from your package.json (or set it to "type": "commonjs").
2. Keep your lint config file named as .eslintrc.js.
3. For your frontend or unit test files that strictly require ES Modules, rename their file extensions from .js to **.mjs**.

Alternatively, if you are using Jest with standard ES Modules, it is often easiest to add a transformer like Babel or ts-jest to compile the code on the fly rather than manually changing extensions

AI-Suggested Content: Proposed a solution for compatibility

Validation: The proposed solution was tested

Decision: Accepted

Reflection: AI helped better understand the module system of the environment.

Responsible Person: Johnny Quach

Task SCRUM-24: Set up the integration testing

Purpose of AI Use: Create configuration files for integration testing and integration test files

Prompt: github repo with client and server structure. already have the pipeline yaml file. setup the integration test files for the client directory in react framework and the server directory in node.js and express.js environment

Response:

To set up integration tests for a monorepo structured with a **client/ (React)** and **server/ (Node.js/Express)** layout, we will use modern industry-standard frameworks: **Playwright** for the React client (handling true end-to-end integration via the UI) and **Supertest** with **Jest** for the Express server (testing HTTP endpoints without binding to a live network port).

1. Backend Integration Setup (server/)
We will use Jest as our runner and Supertest to programmatically make requests to your Express application.
Step A: Install Dependencies

Navigate to your server/ directory and run:

```
cd server
npm install --save-dev jest supertest
```

Step B: Decouple Your Express App

To run clean integration tests, **never export an active server listening on a port**. Export the app instance instead.
server/app.js (or app.ts):

```
const express = require('express');
const app = express();

app.use(express.json());

// Example endpoint
app.get('/api/health', (req, res) => {
  res.status(200).json({ status: 'OK', environment: process.env.NODE_ENV });
});

module.exports = app;
```

server/index.js (Your entry point):

```
const app = require('./app');
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step C: Create the Backend Integration Test

Create a file named app.test.js under an integration directory inside your server folder.

server/tests/integration/app.test.js:

```
const request = require('supertest');
const app = require('../../app'); // Path to your exported express app

describe('Server Integration Tests', () => {

  it('GET /api/health should return a 200 status and correct payload', async () => {
    const response = await request(app)
      .get('/api/health')
      .expect('Content-Type', /json/)
      .expect(200);

    expect(response.body).toEqual({
      status: 'OK',
      environment: 'test'
    });
  });

});
```

Step D: Configure package.json

Add the test execution script to your backend application:

```
"scripts": {
  "test": "NODE_ENV=test jest --runInBand --detectOpenHandles"
}
```

2. Frontend Integration Setup (client/)

For a React client, modern integration testing means asserting that components and API interactions play nicely together. Playwright is perfect for this because it allows you to test user interactions in a headless browser while letting you easily mock server endpoints so the client can be tested in isolation.

Step A: Install Dependencies

Navigate to your client/ directory and initialize Playwright:

```
cd ../client
npm init playwright@latest
```

*During initialization, select your preferences (JavaScript/TypeScript, default tests directory, and whether to add a GitHub Actions workflow—choose **no** since you already have a pipeline file).*

Step B: Create the Frontend Integration Test

Write a test that mocks the backend response and validates that the React UI displays it correctly.

client/tests/integration/ui.spec.js:

```
import { test, expect } from '@playwright/test';

test.describe('React Application Integration', () => {

  test('should mock backend health check and display state on screen', async ({ page }) => {
    // Intercept client API calls and supply a mock backend response
    await page.route('**/api/health', async (route) => {
      const json = { status: 'Mocked OK', environment: 'frontend-test' };
      await route.fulfill({ json });
    });

    // Go to your local development or pipeline application URL
    await page.goto('http://localhost:3000');

    // Assert that your React component updated properly based on the mocked API response
    const statusText = page.locator('#backend-status');
    await expect(statusText).toContainText('Mocked OK');
  });

});
```

Step C: Update Playwright Config

Ensure your Playwright configuration file is set up to find your integration tests folder and optionally spin up the React dev server automatically.

client/playwright.config.js:

```
const { defineConfig, devices } = require('@playwright/test');

module.exports = defineConfig({
```

```
  testDir: './tests/integration',
  fullyParallel: true,
  reporter: 'html',
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
  },
  /* Optional: Run your local dev server before starting the tests */
  webServer: {
    command: 'npm run start',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },
});
```

3. Incorporating Tests into Your Pipeline

Since you already have a configuration pipeline file (like .github/workflows/main.yml), make sure your workflow sequence navigates into the subdirectories and runs both test groups using the matching framework scripts:

```
# Inside your pipeline step blocks:

- name: Run Backend Integration Tests
  run: |
    cd server
    npm ci
    npm test

- name: Run Frontend Integration Tests
  run: |
    cd client
    npm ci
    npx playwright install --with-deps
    npx playwright test
```

AI-Suggested Content: Generated integration test files for the client and server directories and generated a configuration file for the client directory.

Validation: The configuration file was tested while the generated integration test files were compared with requirements.

Decision: Partially accepted. The generated integration test files for the server directory were not used, but the configuration file for integration testing for the client directory was modified before use.

Reflection: AI helped recognize the need to test the server's routing layer and react components.

Responsible Person: Johnny Quach

Task SCRUM-24: Set up integration testing for the client directory

Purpose of AI Use: Create integration test files for the client directory

Prompt: for the frontend, running "npx playwright test" gives the following error "npm error Missing script: "start""

Response:

Since you are using **Vite**, your application boots up using npm run dev instead of npm run start, and Vite serves files on port **5173** by default rather than port 3000.

To fix this, we need to update your **client/playwright.config.js** file so Playwright knows how to start your Vite server and where to look for it.

Update your client/playwright.config.js
Open your Playwright configuration file and update the use and webServer blocks to match Vite's defaults:

```
const { defineConfig, devices } = require('@playwright/test');

module.exports = defineConfig({
  testDir: './tests/integration',
  fullyParallel: true,
  reporter: 'html',
  use: {
    // 1. Update the base URL to match Vite's default local port
    baseURL: 'http://localhost:5173',
    trace: 'on-first-retry',
  },
  /* Run your local dev server before starting the tests */
  webServer: {
    // 2. Change the command from 'npm run start' to 'npm run dev'
    command: 'npm run dev',
    // 3. Update the target URL to port 5173
    url: 'http://localhost:5173',
    reuseExistingServer: !process.env.CI,
    timeout: 120000, // Optional: 2-minute timeout safety cushion for CI environments
  },
});
```

Verify and Run
Now, navigate to your client folder and execute the test runner again:

```
cd client
npx playwright test
```

Pipeline Adjustment 🚀
Don't forget to update your pipeline YAML workflow if it contains any references to port 3000 or the old start scripts. Playwright will now smoothly spin up Vite, intercept your endpoints, and run your UI integration specs completely in isolation.

AI-Suggested Content: Generated a configuration file for integration testing that uses the same technology

Validation: The generated configuration was tested

Decision: Accepted

Reflection: AI helped better understand how integration testing works for the client directory, but more knowledge is required to fully implement it.

Responsible Person: Johnny Quach

Task SCRUM-24: Unit testing error

Purpose of AI Use: To fix the error that occurs when running unit testing

Prompt: github repo for web app with client and server directory. already have the unit tests and the integration tests. there are files jest.config.cjs and jest.config.json. running "npm run test:unit" gives the following error

"Multiple configurations found" and " Implicit config resolution does not allow multiple configuration files. Either remove unused config files or select one explicitly with `--config`."

Response:

You can fix this error by deleting one of the configuration files or by explicitly pointing to one using the --config flag in your package.json script. Jest does not allow multiple implicit configuration files in the same directory.

Solution 1: Delete the unused configuration file (Recommended)
Usually, having both jest.config.cjs and jest.config.json is an accident (e.g., one was generated automatically or left over from a migration).

1. Review both files to see which one contains your actual project settings.
2. Consolidate any unique settings into a single file.
3. **Delete** the other file. Jest prefers JavaScript/CommonJS configurations (jest.config.cjs or jest.config.js) for modern web apps because they allow you to use dynamic logic or environment variables.

Solution 2: Explicitly target a configuration file
If you intentionally need separate configurations for your client and server (or unit and integration tests), you must tell Jest exactly which file to use by updating your package.json scripts.

Modify the test:unit script in your package.json to include the --config flag:

```
"scripts": {
  "test:unit": "jest --config jest.config.json"
}
```

(Replace jest.config.json with jest.config.cjs or the specific path to your unit test config if they are separated).

AI-Suggested Content: Proposed two solutions to fix the issue

Validation: The first proposed solution was tested and reviewed

Decision: Accepted

Reflection: AI helped better understand the script works for unit testing.

Responsible Person: Johnny Quach

Task SCRUM-24: Configure unit and integration testing

Purpose of AI Use: To configure properly unit and integration testing

Prompt: running both "npm run test:unit" and "npm run test:integration" work but they test the same thing. in the server directory, there is a folder called "tests" that contains one integration test file called "server.integration.test.js"

Response:

Your tests are running the exact same files because Jest's default behavior is to search your entire project for any file ending in .test.js or .spec.js. Since your configurations aren't separated, both scripts are picking up both unit and integration tests.

To fix this, you need to use Jest's testMatch or testRegex properties to tell each script exactly where to look.

Here is the cleanest way to separate them using a single **jest.config.cjs** file (you can safely **delete** jest.config.json after doing this).

Step 1: Update your jest.config.cjs

Open your jest.config.cjs file and use Jest **projects** to split the configurations for your unit tests and your server integration tests. Update the file to look like this:

```
module.exports = {
  // Global settings can go here
  projects: [
    {
      displayName: 'unit',
      testMatch: [
        '**/__tests__/**/*.[jt]s?(x)',
        '**/?(*.)+(spec|test).[jt]s?(x)'
      ],
      // Exclude the server integration directory from unit tests
      testPathIgnorePatterns: ['/node_modules/', '/server/tests/'],
    },
    {
      displayName: 'integration',
      // Only run files inside your server/tests folder
      testMatch: [
        '<rootDir>/server/tests/**/*.integration.test.js'
      ],
    }
  ]
};
```

Step 2: Update your package.json scripts
Now, update the scripts in your root package.json to target these specific project configurations using the --selectProjects flag:

```
"scripts": {
  "test:unit": "jest --selectProjects unit",
  "test:integration": "jest --selectProjects integration",
  "test:all": "jest"
}
```

Why this works:

- Running npm run test:unit will now completely ignore the server/tests/ folder.
- Running npm run test:integration will *only* execute the server.integration.test.js file.
- Running npm run test:all (or just jest) will run everything side-by-side.

AI-Suggested Content: Generated a modification to two existing configuration files

Validation: The generated modification was tested and reviewed

Decision: Accepted

Reflection: AI helped better understand how to configure unit and integration testing.

Responsible Person: Johnny Quach

Task SCRUM-24: Modify the integration test file for the server directory

Purpose of AI Use: To configure properly the integration testing for the server

Prompt: github repo for web app with client and server directories. client runs on react vite framework while server runs on node.js with express.js. set up and server.integration.test.js file tailored to this server.js

(The file server.js was shared)

Response:

To create clean, reliable integration tests for your Express app without running into port-clash issues, we use **supertest** paired with an in-memory database helper like **mongodb-memory-server**.
Because your server.js file calls app.listen() directly inside it, importing server.js normally will trigger a live network port binding. The standard industry pattern to fix this is exporting the app instance from server.js so supertest can spin it up dynamically on ephemeral ports.

Step 1: Minor Adjustment to server.js

Open your server.js file and add this export at the very bottom of the file:

```
// Add this line at the bottom of server.js
module.exports = app;
```

Step 2: Install Test Dependencies

Run this command inside your **server directory** to install the testing framework:

```
npm install --save-dev jest supertest mongodb-memory-server
```

Step 3: Create your server.integration.test.js

Create a new file named server.integration.test.js in your test folder (or root of your server folder) and add the following tailored code. It mocks MongoDB in memory, mocks your custom requireAuth middleware for easy profile path testing, and runs assertions against your actual endpoints.

```
const request = require('supertest');
const { MongoMemoryServer } = require('mongodb-memory-server');
const mongoose = require('mongoose');
const jwt = require('jsonwebtoken');

// Set up mandatory environment variables before importing the app
process.env.JWT_SECRET = 'test_secret_key_12345';
process.env.MONGO_URI = 'mongodb://localhost:27017/placeholder'; // Will be overridden

// Mock the authentication middleware so we can easily test protected routes
jest.mock('./middleware/auth', () => {
```

```
  return (req, res, next) => {
    // Look for a test header we pass during supertest calls
    if (req.headers.test_user_id) {
      req.user = { id: req.headers.test_user_id };
      return next();
    }
    return res.status(401).json({ error: 'Unauthorized' });
  };
});

// Import the app instance from server.js
const app = require('./server');
const User = require('./models/User');

let mongoServer;

beforeAll(async () => {
  // Spin up an isolated, in-memory MongoDB server
  mongoServer = await MongoMemoryServer.create();
  const uri = mongoServer.getUri();

  // Disconnect any existing global connections and reconnect to memory DB
  if (mongoose.connection.readyState !== 0) {
    await mongoose.disconnect();
  }
  await mongoose.connect(uri);
});

afterAll(async () => {
  // Cleanup and shut down connection gracefully
  await mongoose.disconnect();
  await mongoServer.stop();
});

beforeEach(async () => {
  // Clear out data between individual test blocks to keep them pure
  const collections = mongoose.connection.collections;
  for (const key in collections) {
    await collections[key].deleteMany();
  }
});

describe('CareerConnect Integration Tests', () => {
```

```
  // 1. Health Check Test
  describe('GET /', () => {
    it('should return a 200 status and running message', async () => {
      const res = await request(app).get('/');
      expect(res.status).toBe(200);
      expect(res.body).toHaveProperty('message', 'CareerConnect Server is running!');
    });
  });

  // 2. User Registration Tests
  describe('POST /api/register', () => {
    it('should register a new user successfully', async () => {
      const res = await request(app)
        .post('/api/register')
        .send({
          email: 'test@example.com',
          password: 'securepassword123'
        });

      expect(res.status).toBe(201);
      expect(res.body).toHaveProperty('message', 'User registered successfully!');
      expect(res.body.user).toHaveProperty('email', 'test@example.com');
      expect(res.body.user).not.toHaveProperty('password'); // Ensure hashed
password isn't leaked
    });

    it('should fail if email or password fields are missing', async () => {
      const res = await request(app)
        .post('/api/register')
        .send({ email: 'test@example.com' }); // missing password

      expect(res.status).toBe(400);
      expect(res.body).toHaveProperty('error', 'Email and password are required');
    });

    it('should fail if email is already taken', async () => {
      // Seed a user into our memory database first
      await request(app)
        .post('/api/register')
        .send({ email: 'duplicate@example.com', password: 'password123' });

      // Attempt to register identical email
      const res = await request(app)
        .post('/api/register')
```

```
            .send({ email: 'duplicate@example.com', password: 'differentpassword' });

        expect(res.status).toBe(409);
        expect(res.body).toHaveProperty('error', 'Email already registered');
      });
    });

    // 3. Login Tests
    describe('POST /api/login', () => {
      beforeEach(async () => {
        // Create a default user profile to test logins against
        await request(app)
          .post('/api/register')
          .send({ email: 'loginme@example.com', password: 'correct_password' });
      });

      it('should return a JWT token on valid credentials', async () => {
        const res = await request(app)
          .post('/api/login')
          .send({ email: 'loginme@example.com', password: 'correct_password' });

        expect(res.status).toBe(200);
        expect(res.body).toHaveProperty('token');
        expect(res.body).toHaveProperty('message', 'Login successful!');
      });

      it('should reject invalid passwords with 401', async () => {
        const res = await request(app)
          .post('/api/login')
          .send({ email: 'loginme@example.com', password: 'wrong_password' });

        expect(res.status).toBe(401);
        expect(res.body).toHaveProperty('error', 'Invalid email or password');
      });
    });

    // 4. Authenticated Profile Tests
    describe('GET /api/profile and PUT /api/profile', () => {
      let userId;

      beforeEach(async () => {
        // Create user profile directly via Mongoose model to secure a real system Object
ID
        const user = await User.create({
```

```
      email: 'profileuser@example.com',
      password: 'password123',
      name: 'Jane Doe',
      role: 'job_seeker'
    });
    userId = user._id.toString();
  });

  it('should fetch the correct user profile when authenticated', async () => {
    const res = await request(app)
      .get('/api/profile')
      .set('test_user_id', userId); // Passes mock auth header verification

    expect(res.status).toBe(200);
    expect(res.body.user).toHaveProperty('email', 'profileuser@example.com');
    expect(res.body.user).toHaveProperty('name', 'Jane Doe');
  });

  it('should successfully update valid fields on PUT', async () => {
    const res = await request(app)
      .put('/api/profile')
      .set('test_user_id', userId)
      .send({
        email: 'newemail@example.com',
        name: 'Jane Smith',
        role: 'recruiter',
        location: 'Montreal',
        bio: 'Experienced developer.'
      });

    expect(res.status).toBe(200);
    expect(res.body.user.email).toBe('newemail@example.com');
    expect(res.body.user.role).toBe('recruiter');
    expect(res.body.user.location).toBe('Montreal');
  });

  it('should reject invalid values (like wrong roles)', async () => {
    const res = await request(app)
      .put('/api/profile')
      .set('test_user_id', userId)
      .send({
        email: 'profileuser@example.com',
        role: 'invalid_role_type' // Must be job_seeker or recruiter
      });
```

```
        expect(res.status).toBe(400);
        expect(res.body).toHaveProperty('error', 'Invalid role');
      });
    });
});
```

AI-Suggested Content: Generated an integration test file specific to the server.js

Validation: The generated integration test file was tested with scripts

Decision: Accepted

Reflection: AI helped better understand how integration testing works on the server directory.

Responsible Person: Johnny Quach

Task SCRUM-24: Lint error in the client directory

Purpose of AI Use: To fix the lint error one the client side

Prompt: this is the file that contains the lint error

(The file ResumeManager.jsx was shared)

Response:

The lint error occurs because loadResume is a synchronous function wrapper that immediately invokes state setters (setLoading(true), setError('')) when called inside your useEffect hook. Even though the inner network request is asynchronous, ESLint flags the synchronous execution at the top of the function during the rendering cycle.

To fix this, you should **move the loadResume definition inside the useEffect hook** or rewrite it to cleanly isolate the asynchronous flow.

The Fixed Code

Here is the corrected file. The loadResume function has been moved directly inside the useEffect block, which fixes the linting error and lets you safely remove the eslint-disable-next-line comment.

```
import { useEffect, useRef, useState } from 'react';
import {
  fetchResumeMetadata,
  fetchResumeBlobUrl,
  uploadResume,
  replaceResume,
  deleteResume,
} from '../api/resumeApi';
import { useAuth } from '../context/AuthContext';

const ALLOWED_EXTENSIONS = ['.pdf', '.doc', '.docx'];
const MAX_FILE_SIZE_MB = 5;

function formatFileSize(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`;
}

function ResumeManager() {
  const { token, isAuthenticated } = useAuth();
  const [resume, setResume] = useState(null);
  const [loading, setLoading] = useState(false);
```

```
const [error, setError] = useState('');
const fileInputRef = useRef(null);

// Keep useEffect logic fully contained to avoid cascading render flags
useEffect(() => {
  const loadResume = async () => {
    if (!token) {
      setResume(null);
      return;
    }
    setLoading(true);
    setError('');
    try {
      const data = await fetchResumeMetadata(token);
      setResume(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  loadResume();
}, [token]);

const validateFile = (file) => {
  const extension = file.name
    .slice(file.name.lastIndexOf('.'))
    .toLowerCase();
  if (!ALLOWED_EXTENSIONS.includes(extension)) {
    return 'Only PDF, DOC, and DOCX files are allowed.';
  }
  if (file.size > MAX_FILE_SIZE_MB * 1024 * 1024) {
    return `File is too large. Maximum size is ${MAX_FILE_SIZE_MB}MB.`;
  }
  return null;
};

const handleFileChange = async (event) => {
  const file = event.target.files?.[0];
  event.target.value = ''; // allow re-selecting the same file later
  if (!file || !token) return;

  const validationError = validateFile(file);
```

```
  if (validationError) {
    setError(validationError);
    return;
  }

  setLoading(true);
  setError('');
  try {
    const data = resume
      ? await replaceResume(token, file)
      : await uploadResume(token, file);
    setResume(data);
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
};

const handleDelete = async () => {
  if (!token || !resume) return;
  if (!window.confirm('Delete your uploaded resume?')) return;

  setLoading(true);
  setError('');
  try {
    await deleteResume(token);
    setResume(null);
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
};

const handleViewOrDownload = async (download) => {
  if (!token) return;
  setError('');
  try {
    const blobUrl = await fetchResumeBlobUrl(token, { download });
    const link = document.createElement('a');
    link.href = blobUrl;
    if (download) {
      link.download = resume?.originalName || 'resume';
```

```
    } else {
      link.target = '_blank';
      link.rel = 'noopener noreferrer';
    }
    document.body.appendChild(link);
    link.click();
    link.remove();
    setTimeout(() => URL.revokeObjectURL(blobUrl), 10000);
  } catch (err) {
    setError(err.message);
  }
};

return (
  <section className="resume-manager-section" id="resume">
    <h3>My Resume</h3>

    {!isAuthenticated && (
      <p className="resume-hint">Log in to upload and manage your resume.</p>
    )}

    {error && <p className="resume-error">{error}</p>}

    {isAuthenticated && loading && <p>Loading...</p>}

    {isAuthenticated && !loading && resume && (
      <div className="resume-card">
        <div>
          <strong>{resume.originalName}</strong>
          <div className="resume-meta">
            {formatFileSize(resume.size)}
          </div>
        </div>
        <div className="resume-actions">
          <button type="button" onClick={() => handleViewOrDownload(false)}>
            View
          </button>
          <button type="button" onClick={() => handleViewOrDownload(true)}>
            Download
          </button>
          <button type="button" onClick={() => fileInputRef.current?.click()}>
            Replace
          </button>
          <button type="button" onClick={handleDelete}>
```

```
          Delete
        </button>
      </div>
    </div>
  )}

  {isAuthenticated && !loading && !resume && !error && (
    <button type="button" onClick={() => fileInputRef.current?.click()}>
      Upload Resume
    </button>
  )}

  <input
    ref={fileInputRef}
    type="file"
    accept=".pdf,.doc,.docx"
    onChange={handleFileChange}
    style={{ display: 'none' }}
  />
 </section>
 );
}

export default ResumeManager;
```

Why this fixes the issue

- **Scope Isolation:** Defining loadResume inside the hook explicitly tells React's static analysis tools that the function is an implementation detail of that specific side-effect.
- **Dependency Accuracy:** This satisfies the react-hooks/exhaustive-deps rule natively without needing a comment override, making your codebase cleaner and less error-prone moving forward.

AI-Suggested Content: Generated a modification to the existing component file

Validation: Tested the modification

Decision: Accepted

Reflection: AI helped understand the importance of the modification.

Responsible Person: Johnny Quach

Task SCRUM-24: Integration test error on the server

Purpose of AI Use: To fix the integration testing error

Prompt: running "npm run test:integration" gives the following

"Jest has detected the following 1 open handle potentially keeping Jest from exiting:

● TCPSERVERWRAP 176 | }); 177 | > 178 | app.listen(PORT, () => { | ^ 179 | console.log(`Server listening on http://localhost:${PORT}`); 180 | }); 181 |"

Response:

The error happens because **app.listen() inside your server.js file is still spinning up a live network server**, even though supertest is already managing an isolated instance for your tests. Because Jest has no way of knowing how to shut down that background server loop, the process hangs.

You can fix this easily by **conditionally starting the server** so it only runs when server.js is executed directly, and skips the listener when imported by Jest.

The Fix

Open your **server.js** file, locate the app.listen() block at the bottom, and wrap it in a condition checking if it's being run directly by Node vs. imported by a test framework:

```
// ❌ REPLACE THIS OLD BLOCK:
// app.listen(PORT, () => {
//     console.log(`Server listening on http://localhost:${PORT}`);
// });

//  WITH THIS CONDITIONALLY GATED BLOCK:
if (process.env.NODE_ENV !== 'test') {
   app.listen(PORT, () => {
      console.log(`Server listening on http://localhost:${PORT}`);
   });
}

module.exports = app;
```

Why this fixes the open handle

- **During normal app usage (npm start):** process.env.NODE_ENV is not set to 'test', so your server starts up perfectly on your designated port.
- **During testing (npm run test:integration):** Jest automatically injections process.env.NODE_ENV = 'test'. The conditional block completely bypasses the listener, allowing supertest to safely mount your app configuration onto its own short-lived, self-closing ports.

AI-Suggested Content: Generated a modification to the existing server.js file

Validation: Tested the modification

Decision: Accepted

Reflection: AI helped better understand the importance of the modification and how it works.

Responsible Person: Johnny Quach

Task SCRUM-24: Pipeline integration test script

Purpose of AI Use: To make the pipeline run the integration test

Prompt: how to make the pipeline work when running the integration test for the mongodb server

Response:

To make your integration tests work inside a CI/CD pipeline, you must configure the pipeline YAML to spin up an ephemeral, isolated MongoDB instance alongside your application.

Here are the standard configurations for the most common CI/CD platforms.

Option 1: GitHub Actions (.github/workflows/test.yml)
GitHub Actions uses **Service Containers** to run database sidecars.

```
name: Integration Tests

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest

    # 1. Spin up MongoDB as a service container
    services:
      mongodb:
        image: mongo:latest
        ports:
          - 27017:27017

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node/Python/Java
        uses: actions/setup-node@v4 # (or your specific language setup)
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm install

      # 2. Run your tests pointing to localhost:27017
```

```
    - name: Run Integration Tests
      run: npm test
      env:
        MONGO_URI: mongodb://localhost:27017/test_db
```

Option 2: GitLab CI/CD (.gitlab-ci.yml)

GitLab uses the **services** keyword. Because of how GitLab networks containers, you usually connect via the hostname mongodb instead of localhost.

```
image: node:20 # Your application runtime image

# 1. Define the MongoDB service
services:
  - name: mongo:latest
    alias: mongodb

variables:
  # 2. Point your app to the service's network alias
  MONGO_URI: "mongodb://mongodb:27017/test_db"

stages:
  - test

integration_tests:
  stage: test
  script:
    - npm install
    - npm test
```

Option 3: Docker Compose (docker-compose.yml)

If your pipeline relies on Docker Compose, you network the containers together under the same bridge network.

```
version: '3.8'

services:
  # 1. The Application container
  app:
    build: .
    command: npm test
    environment:
      - MONGO_URI=mongodb://mongo_db:27017/test_db
    depends_on:
      - mongo_db
```

```
  # 2. The MongoDB container
  mongo_db:
    image: mongo:latest
    ports:
      - "27017:27017"
```

💡 **Two Crucial Rules for Your Test Code**

1. **Use Environment Variables:** Never hardcode mongodb://localhost:27017 in your source code. Use a variable like process.env.MONGO_URI or os.environ["MONGO_URI"] so your pipeline can change the target address depending on its platform.
2. **Handle Startup Delays:** Databases take a few seconds to start up and allocate data files. Ensure your test suite has a "retry/wait" connection loop, or use a pipeline tool like dockerize -wait tcp://localhost:27017 to prevent your tests from running before MongoDB is ready.

AI-Suggested Content: Generated three configuration files for the pipeline

Validation: Tested and reviewed the first generated configuration file for the pipeline

Decision: Accepted

Reflection: AI helped better understand the importance of setting up a container to run the database server.

Responsible Person: Johnny Quach